A SYNOPSIS ON

REAL-TIME MESSENGER APPLICATION

Submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF COMPUTER APPLICATION

Submitted by:

Megha Dhami	2471126
Hridyansh Sati	2471085
Hrithik Bashera	2471086

Under the Guidance of

Mr. Janmejay Pant

Assistant Professor

Group No. = 24



School of Computing

Graphic Era Hill University, Bhimtal, Uttarakhand

Sep-2026

Graphic Era Hill University
BHIMTAL CAMPUS

CANDIDATE'S DECLARATION

I/We hereby certify that the work which is being presented in the Synopsis entitled "Real-Time Messenger Application" in partial fulfilment of the requirements for the award of the Degree of Bachelor Computer Application of the Graphic Era Hill University, Bhimtal shall be carried out by the undersigned under the supervision of Mr. Janmejay Pant, School of Computing, Graphic Era Hill University, Bhimtal.

Megha Dhami	2471126	signature
Hridyansh Sati	2471085	signature
Hrithik Bashera	2471086	signature

The above mentioned students shall be working under the supervision of the undersigned on the "Real-Time Messenger Application"

Signature	Signature
Supervisor	Head of the Department

Internal Evaluation (By DPRC Committee)

Status of the Synopsis: Accepted / Rejected

Any Comments:

Name of the Committee Members: Signature with Date

1.

2.

Table of Contents

Chapter No.	Description	Page No
Chapter 1	Introduction and Problem Statement	1-3
Chapter 2	Background/ Literature Survey	4-6
Chapter 3	Objectives	7-10
Chapter 4	System Architecture	11
Chapter 5	Hardware and Software Requirements	12-13
Chapter 6	Possible Approach/ Algorithms	14-15
Chapter 7	Conclusion and references	16

Chapter 1 – Introduction and Problem Statement

1.1 Introduction

Communication has become an essential part of everyday life. People need fast and convenient ways to communicate with friends, classmates, colleagues, and other users. Traditional communication methods may not provide features such as instant messaging, message history, online status, and media sharing in a single application.

The Real-Time Messenger Application Using Java is designed to provide a simple and efficient platform for real-time communication between users. The application allows users to register and log in securely, view other registered users, and communicate through one-to-one messaging.

The project uses Java as the primary programming language. JavaFX is used to develop the desktop user interface, while Spring Boot is used for backend development and REST APIs. WebSocket technology is used to support real-time communication, allowing messages to be delivered without continuously refreshing the application. MySQL is used to store user information and message history.

The system is divided into different modules, including user authentication, user management, real-time chat, message history, online/offline status, and media sharing. The project demonstrates the practical application of Java, object-oriented programming, database management, client-server communication, and real-time communication technologies.

1.2 Problem Statement

In today's digital environment, users require a communication system that is fast, reliable, easy to use, and capable of providing real-time interaction. Developing a messaging application involves several challenges, such as securely managing users, storing conversations, delivering messages in real time, and maintaining the communication status of users.

Basic messaging systems may have limitations such as:

- Lack of real-time message delivery.
- No proper user authentication and account management.
- Difficulty in maintaining and retrieving previous conversations.
- Lack of online/offline user status.
- Limited support for sharing media through conversations.
- Poor integration between the user interface, backend, and database.

Therefore, there is a need for a centralized messaging application that can provide real-time one-to-one communication, maintain message history, manage users securely, and provide a simple user-friendly interface.

The proposed Real-Time Messenger Application Using Java aims to address these problems by integrating a JavaFX frontend, Spring Boot backend, WebSocket-based communication, and MySQL database into a single system.

Chapter 2 - Background / Literature Survey

2.1 Background

The rapid growth of digital communication has increased the need for messaging applications that can provide fast and reliable communication between users. Messaging systems have evolved from simple text-based communication to platforms that support real-time messaging, message history, user status, and media sharing.

A real-time messenger application generally follows a client-server architecture, where the client provides the user interface and the server manages users, messages, authentication, and communication between clients. A database is used to store important information such as user details and previous messages.

For the proposed project, Java is used as the primary programming language. JavaFX provides the desktop-based user interface, while Spring Boot is used to develop the backend and REST APIs. WebSocket is used for real-time communication so that messages can be exchanged between users without repeatedly refreshing the application. MySQL is used for storing user and messaging-related data.

The project combines these technologies to develop a communication system that is modular, maintainable, and suitable for real-time one-to-one messaging.

2.2 Literature Survey

Several existing messaging and communication systems demonstrate the importance of real-time communication in modern applications. Common features found in such systems include user authentication, one-to-one messaging, message history, online/offline status, and media sharing.

Traditional messaging systems based mainly on repeated requests to a server can introduce delays because the client needs to continuously check whether new messages are available. Real-time communication technologies such as WebSocket provide a more suitable approach by maintaining a continuous communication channel between the client and server.

Research and existing implementations of Java-based communication systems show that Java provides useful features for developing network-based applications. Its object-oriented structure, exception handling, database connectivity, and extensive frameworks make it suitable for developing applications with multiple interacting modules.

Spring Boot simplifies backend development by providing support for REST APIs, application configuration, database integration, and server-side components. JavaFX provides components for developing desktop interfaces, making it suitable for creating screens such as login, registration, user lists, and chat windows.

The literature and existing systems indicate that combining Java, JavaFX, Spring Boot, WebSocket, and MySQL can provide an effective foundation for developing a real-time messenger application. The proposed project focuses on integrating these technologies into one system while keeping the application simple and user-friendly.

2.3 Existing System

Existing messaging applications generally provide features such as:

- User registration and login
- One-to-one and group communication
- Real-time message delivery
- Message history
- Online/offline status
- Media sharing
- Notifications

However, large commercial messaging platforms often contain many advanced features that are not necessary for a basic academic project. Their complex architecture can also make it difficult for students to understand the complete implementation.

2.4 Proposed System

The proposed Real-Time Messenger Application Using Java focuses on the core requirements of a messaging system.

The proposed system will provide:

- Secure user registration and login
- User management
- One-to-one real-time messaging
- Message history
- Online/offline status
- Photo/media sharing
- JavaFX-based desktop interface
- MySQL database for data storage
- Spring Boot backend and REST APIs
- WebSocket-based real-time communication

The proposed system is intended to provide a practical implementation of a real-time communication application while demonstrating the use of Java and related technologies.

Chapter 3 - Objectives

The main objective of the Real-Time Messenger Application Using Java is to develop a reliable and user-friendly communication system that enables users to exchange messages in real time. The project aims to combine Java-based technologies with database and networking concepts to create an effective messaging application.

3.1 Main Objectives

1. To develop a real-time messaging application that allows users to communicate through one-to-one chats.
2. To provide secure user authentication through registration and login functionality.
3. To implement real-time message communication using WebSocket technology.
4. To store and manage user information and message history using a MySQL database.
5. To develop an interactive desktop interface using JavaFX for easy navigation and communication.
6. To provide online and offline user status so that users can identify the availability of other users.
7. To support photo/media sharing between users as an additional communication feature.
8. To develop REST APIs using Spring Boot for communication between the frontend and backend.
9. To apply Java Object-Oriented Programming concepts such as classes, objects, encapsulation, inheritance, and polymorphism wherever applicable.
10. To create a modular and maintainable system in which different components such as authentication, chat, database, and user management can work together efficiently.
11. To provide message history so users can access previously exchanged messages.
12. To gain practical experience in Java application development, database connectivity, client-server architecture, and real-time communication technologies.

Chapter 4 - System Architecture and Algorithms

4.1 System Architecture

The **Real-Time Messenger Application Using Java** follows a **client-server architecture**. The system consists of three main layers: the frontend, backend, and database. These layers work together to provide authentication, real-time messaging, data storage, and user management.

Main Components

1. Client / Frontend Layer

The frontend is developed using **JavaFX**. It provides the interface through which users can:

- Register and log in.
- View available users.
- Open one-to-one conversations.
- Send and receive messages.
- View message history.
- Check online/offline status.
- Share photos or media.

2. Backend Layer

The backend is developed using **Java Spring Boot**. It handles:

- User authentication.
- User management.
- REST API requests.
- Message processing.
- Database operations.
- Communication between different components of the application.

3. Real-Time Communication Layer

WebSocket is used to enable real-time communication between users. When a user sends a message, the server can deliver it to the intended receiver without requiring the receiver to continuously refresh the application.

4. Database Layer

MySQL is used to store and manage application data such as:

- User details
- Login-related information
- Messages
- Message timestamps
- User status
- Media-related information

4.2 Working of the System

The overall working of the system can be described as follows:

1. The user opens the Messenger application.
2. The user registers an account or logs into an existing account.
3. The frontend sends the required request to the Spring Boot backend.
4. The backend verifies the user's information using the MySQL database.
5. After successful login, the user can view the list of available users.

6. The user selects another user and opens a chat.
7. The message is sent through the WebSocket connection.
8. The backend processes the message and delivers it to the intended user in real time.
9. The message is stored in the database for future access.
10. The receiver can view the message immediately and access previous messages through chat history.

4.3 Algorithms

Algorithm 1 – User Registration
Input: Username, email, password
Output: Successful registration or error message
Step 1: Start
Step 2: Enter username, email and password.
Step 3: Validate the entered information.
Step 4: Check whether the email/username already exists.
Step 5: If it exists, display "User already exists".
Step 6: Otherwise, store the user information in the database.
Step 7: Display "Registration Successful".
Step 8: Stop

Algorithm 2 – User Login
Input: Email/Username and Password
Output: Login success or failure
Step 1: Start
Step 2: Enter username/email and password.
Step 3: Send login details to the backend.
Step 4: Check the entered credentials in the database.
Step 5: If the credentials are correct, allow the user to enter the application.
Step 6: Otherwise, display "Invalid username or password".
Step 7: Stop

Algorithm 3 – Send Message
Input: Sender ID, Receiver ID, Message
Output: Message delivered to receiver
Step 1: Start
Step 2: Sender selects a receiver.
Step 3: Sender enters the message.
Step 4: Check whether the message is valid.
Step 5: Send the message through WebSocket.
Step 6: Backend receives and processes the message.
Step 7: Deliver the message to the receiver.
Step 8: Store the message in the MySQL database.
Step 9: Display the message in the chat window.
Step 10: Stop

Algorithm 4 – Retrieve Message History
Input: Sender ID and Receiver ID
Output: Previous conversation
Step 1: Start
Step 2: User opens a conversation.
Step 3: Send sender and receiver IDs to the backend.
Step 4: Search the database for messages exchanged between the two users.
Step 5: Retrieve the messages in chronological order.
Step 6: Display the messages in the chat window.
Step 7: Stop

Algorithm 5 – Online/Offline Status
Step 1: Start
Step 2: User logs into the application.
Step 3: Update the user's status as "Online".
Step 4: When the user disconnects or logs out, update the status as "Offline".
Step 5: Display the current status to other users.
Step 6: Stop

Chapter 5– Hardware and Software Requirements

5.1 Hardware Requirements

The hardware requirements for developing and running the Real-Time Messenger Application Using Java are as follows:

Hardware Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB minimum
Hard Disk / SSD	10 GB free space
Display	1366 × 768 resolution
Keyboard	Standard keyboard
Mouse	Standard mouse
Network	Internet/LAN connection

The system can run on a standard computer or laptop with the above configuration. For smoother development and execution, 8 GB RAM or higher is recommended.

5.2 Software Requirements

The software requirements for the proposed system are:

Software	Requirement
Operating System	Windows 10/11, Linux, or macOS
Programming Language	Java
Frontend Technology	JavaFX
Backend Framework	Spring Boot
Database	MySQL
Database Connectivity	JPA/Hibernate
Communication Technology	WebSocket
API	REST API
IDE	IntelliJ IDEA / Eclipse / NetBeans
JDK	JDK 17 or above
Build Tool	Maven
Browser	Google Chrome / Microsoft Edge
Version Control	Git / GitHub

Chapter 6 – Possible Approach / Algorithms

6.1 Possible Approach

The proposed Real-Time Messenger Application Using Java will be developed using a client-server approach. The application will be divided into different modules so that each component can be developed and tested independently.

The development approach will include the following steps:

1. Requirement Analysis
   Identify the basic requirements of the messenger system, such as user registration, login, real-time messaging, message history, online/offline status, and media sharing.

2. System Design
   Design the overall client-server architecture, database structure, user interface, and communication flow.

3. Frontend Development
   Develop the desktop interface using JavaFX, including login, registration, user list, and chat screens.

4. Backend Development
   Develop backend services using Java Spring Boot to handle authentication, users, messages, and database operations.

5. Database Implementation
   Create MySQL tables for storing user details, messages, and other required information.

6. Real-Time Communication
   Implement WebSocket to allow messages to be transmitted between users in real time.

7. Integration
   Connect the JavaFX frontend with the Spring Boot backend and integrate the backend with MySQL.

8. Testing
   Test individual modules and the complete system to identify and fix errors.

9. Deployment and Maintenance
   Run the completed application in the required environment and make improvements based on testing and user requirements.

Chapter 7 – Conclusion and References

7.1 Conclusion

The **Real-Time Messenger Application Using Java** is designed to provide a simple, reliable, and user-friendly platform for real-time communication. The application enables users to register and log in, view other users, exchange messages, access previous conversations, and check online/offline status.
The project combines **Java, JavaFX, Spring Boot, WebSocket, and MySQL** to implement the different components of the system. Java and Spring Boot are used for application and backend development, JavaFX is used to create the desktop interface, WebSocket enables real-time communication, and MySQL is used for storing user and message data.
The development of this project will also provide practical understanding of **Object-Oriented Programming, client-server architecture, database connectivity, REST APIs, and real-time communication**. The modular design of the application allows additional features to be incorporated in the future.
Overall, the proposed system aims to demonstrate how Java-based technologies can be integrated to develop a functional real-time messaging application.

7.2 References

The following resources can be referred to during the development of the project:

1. **Oracle Java Documentation** – Java programming language and API documentation.
2. **Spring Boot Documentation** – Reference for developing Java-based backend applications and REST APIs.
3. **OpenJFX Documentation** – Reference for JavaFX application and user-interface development.
4. **MySQL Documentation** – Reference for database creation, SQL queries, and database management.
5. **Spring WebSocket Documentation** – Reference for implementing WebSocket-based real-time communication.
6. **Hibernate Documentation** – Reference for object-relational mapping and database interaction using JPA/Hibernate.
7. **Maven Documentation** – Reference for project dependency management and application build configuration.